

FILE WITH CODE FOR HOME WORK 4

```
# Load your data

cat("Please select your CSV file...\n")

data <- read.csv(file.choose(), stringsAsFactors = FALSE)


# Display basic info

cat("\n===== \n")

cat("DATA OVERVIEW\n")

cat("===== \n")

cat("Rows:", nrow(data), "\n")

cat("Columns:", ncol(data), "\n")

cat("\nColumn names:\n")

print(names(data))


# =====

# DEFINE VARIABLE GROUPS

# =====


# Hormone variables (for logistic regression)

hormone_vars <- c("hormone1", "hormone2", "hormone3", "hormone4",
                 "hormone5", "hormone6", "hormone7", "hormone8",
                 "hormone10_generated")
```

```

# Other variables (for correlation and regression)

lipid_vars <- c("lipids1", "lipids2", "lipids3", "lipids4", "lipids5")

perox_vars <- c("lipid_pero1", "lipid_pero2", "lipid_pero3", "lipid_pero4",
"lipid_pero5")

antiox_vars <- c("antioxidant1", "antioxidant2", "antioxidant3", "antioxidant4",
"antioxidant5")

other_vars <- c("carb_metabolism", "factor_eth", "factor_h", "factor_pcos",
"factor_pr1")


# Keep only variables that exist in the data

hormone_vars <- hormone_vars[hormone_vars %in% names(data)]

lipid_vars <- lipid_vars[lipid_vars %in% names(data)]

perox_vars <- perox_vars[perox_vars %in% names(data)]

antiox_vars <- antiox_vars[antiox_vars %in% names(data)]

other_vars <- other_vars[other_vars %in% names(data)]


# Combine all non-hormone variables for correlation

all_other_vars <- unique(c(lipid_vars, perox_vars, antiox_vars, other_vars))


cat("\n===== \n")

cat("VARIABLES FOUND IN DATA\n")

cat("===== \n")

```

```
cat("Hormones:", length(hormone_vars), "-", paste(hormone_vars, collapse=" "),
"\n")
```

```
cat("Lipids:", length(lipid_vars), "-", paste(lipid_vars, collapse=" "), "\n")
```

```
cat("Lipid Peroxidation:", length(perox_vars), "-", paste(perox_vars, collapse=" "),
"\n")
```

```
cat("Antioxidants:", length(antiox_vars), "-", paste(antiox_vars, collapse=" "), "\n")
```

```
cat("Other variables:", length(other_vars), "-", paste(other_vars, collapse=" "),
"\n")
```

```
# =====
```

```
# PART 1: CORRELATION ANALYSIS WITH PERMUTATION TESTS
```

```
# =====
```

```
cat("\n\n=====\\n")
```

```
cat("PART 1: CORRELATION ANALYSIS\\n")
```

```
cat("=====\\n")
```

```
# Function to calculate correlation with permutation p-value
```

```
cor_with_permutation <- function(x, y, permutations = 1000) {
```

```
  # Remove missing values
```

```
  complete <- complete.cases(x, y)
```

```
  x <- x[complete]
```

```
  y <- y[complete]
```

```
if (length(x) < 5) return(c(NA, NA, length(x)))
```

```
# Original correlation (Spearman for robustness)
```

```
original_cor <- cor(x, y, method = "spearman")
```

```
# Permutation test
```

```
set.seed(123)
```

```
perm_cors <- numeric(permutations)
```

```
for (i in 1:permutations) {
```

```
  perm_cors[i] <- cor(x, sample(y), method = "spearman")
```

```
}
```

```
# Two-tailed p-value
```

```
p_value <- mean(abs(perm_cors) >= abs(original_cor))
```

```
return(c(original_cor, p_value, length(x)))
```

```
}
```

```
# Create correlation table for all other variables
```

```
cat("\nCalculating correlations between all other variables...\n")
```

```
correlation_results <- data.frame(
```

```
  Variable1 = character(),
```

```

Variable2 = character(),
N = integer(),
Correlation = numeric(),
P_value = numeric(),
Significance = character(),
stringsAsFactors = FALSE
)

# Calculate correlations for all pairs
for (i in 1:(length(all_other_vars)-1)) {
  for (j in (i+1):length(all_other_vars)) {
    result <- cor_with_permutation(data[[all_other_vars[i]]],
data[[all_other_vars[j]]])

    # Determine significance level
    sig <- ""
    if (!is.na(result[2])) {
      if (result[2] < 0.001) sig <- "****"
      else if (result[2] < 0.01) sig <- "***"
      else if (result[2] < 0.05) sig <- "**"
    }

    correlation_results <- rbind(correlation_results, data.frame(

```

```

    Variable1 = all_other_vars[i],
    Variable2 = all_other_vars[j],
    N = result[3],
    Correlation = round(result[1], 4),
    P_value = round(result[2], 6),
    Significance = sig,
    stringsAsFactors = FALSE
  ))
}
}

# Remove NA rows
correlation_results <- correlation_results[!is.na(correlation_results$Correlation), ]

# Sort by absolute correlation
correlation_results <- correlation_results[order(-
abs(correlation_results$Correlation)), ]

# Display results
cat("\nTop 30 strongest correlations:\n")
print(head(correlation_results, 30))

# Save full correlation table

```

```
write.csv(correlation_results, "1_correlation_results_all.csv", row.names = FALSE)
cat("\n✓ Full correlation table saved to '1_correlation_results_all.csv'\n")
```

```
# Create correlation matrix heatmap data
```

```
cat("\nCreating correlation matrix summary...\n")
```

```
# Summary by correlation strength
```

```
strong_cors <- correlation_results[abs(correlation_results$Correlation) >= 0.5, ]
```

```
moderate_cors <- correlation_results[abs(correlation_results$Correlation) >= 0.3
& abs(correlation_results$Correlation) < 0.5, ]
```

```
weak_cors <- correlation_results[abs(correlation_results$Correlation) < 0.3, ]
```

```
cat("\nCorrelation Strength Summary:\n")
```

```
cat(" Strong ( $|r| \geq 0.5$ ):", nrow(strong_cors), "pairs\n")
```

```
cat(" Moderate ( $0.3 \leq |r| < 0.5$ ):", nrow(moderate_cors), "pairs\n")
```

```
cat(" Weak ( $|r| < 0.3$ ):", nrow(weak_cors), "pairs\n")
```

```
# Significant correlations ( $p < 0.05$ )
```

```
sig_cors <- correlation_results[correlation_results$P_value < 0.05, ]
```

```
cat(" Significant ( $p < 0.05$ ):", nrow(sig_cors), "pairs\n")
```

```
# =====
```

```
# PART 2: REGRESSION ANALYSIS WITH BIC SELECTION
```

```
# =====
```

```
cat("\n\n=====\\n")
```

```
cat("PART 2: REGRESSION ANALYSIS\\n")
```

```
cat("=====\\n")
```

```
# Function to find best regression model using BIC
```

```
find_best_regression_model <- function(data, predictors, response) {
```

```
  # Remove missing values
```

```
  complete_data <- data[complete.cases(data[, c(predictors, response)]), ]
```

```
  if (nrow(complete_data) < 10) {
```

```
    return(list(model = NULL, error = "Insufficient data (<10 complete cases)"))
```

```
  }
```

```
  # Limit predictors if too many
```

```
  max_predictors <- min(length(predictors), floor(nrow(complete_data)/5))
```

```
  if (length(predictors) > max_predictors) {
```

```
    # Select predictors with highest correlation to response
```

```
    cors <- sapply(predictors, function(p) {
```

```
      cor(complete_data[[response]], complete_data[[p]], use = "complete.obs")
```

```
    })
```

```
    predictors <- names(sort(abs(cors), decreasing = TRUE))[1:max_predictors]
```



```

}

# Try full model first
if (length(predictors) >= 1) {
  formula_all <- as.formula(paste(response, "~", paste(predictors, collapse = " +
  "")))

  full_model <- tryCatch(lm(formula_all, data = complete_data), error =
function(e) NULL)

  if (!is.null(full_model)) {
    # Stepwise selection using BIC
    best_model <- step(full_model, direction = "both", trace = 0,
      k = log(nrow(complete_data)))
    return(list(model = best_model, error = NULL))
  }
}

# If full model fails, try simple models
best_bic <- Inf
best_model <- NULL

for (pred in predictors) {
  formula <- as.formula(paste(response, "~", pred))

```

```
model <- tryCatch(lm(formula, data = complete_data), error = function(e)
NULL)
```

```
  if (!is.null(model)) {
    bic <- BIC(model)
    if (bic < best_bic) {
      best_bic <- bic
      best_model <- model
    }
  }
}
```

```
if (!is.null(best_model)) {
  return(list(model = best_model, error = NULL))
} else {
  return(list(model = NULL, error = "No model could be fitted"))
}
}
```

```
# Analyze each variable group
regression_summary <- data.frame(
  Response_Variable = character(),
  Variable_Type = character(),
```

```

R_squared = numeric(),
Adj_R_squared = numeric(),
BIC = numeric(),
AIC = numeric(),
Num_Predictors = integer(),
Predictors = character(),
Best_Predictor = character(),
Best_Predictor_Coef = numeric(),
Best_Predictor_Pvalue = numeric(),
stringsAsFactors = FALSE
)

```

Function to analyze a group of response variables

```

analyze_response_group <- function(data, response_vars, var_type, all_predictors)
{
  results <- list()

  for (response in response_vars) {
    cat("\n-----\n")
    cat("Analyzing:", response, "(", var_type, ")\n")
    cat("-----\n")

    # Use all other variables as predictors

```

```
predictors <- all_predictors[all_predictors != response]
```

```
if (length(predictors) == 0) {  
  cat(" No predictors available\n")  
  next  
}
```

```
result <- find_best_regression_model(data, predictors, response)
```

```
if (!is.null(result$model)) {  
  model <- result$model  
  summary_model <- summary(model)
```

```
# Get coefficients
```

```
coefs <- coef(summary_model)
```

```
# Find best predictor (largest absolute coefficient excluding intercept)
```

```
if (nrow(coefs) > 1) {
```

```
  coefs_no_intercept <- coefs[-1, , drop = FALSE]
```

```
  if (nrow(coefs_no_intercept) > 0) {
```

```
    abs_coefs <- abs(coefs_no_intercept[, 1])
```

```
    best_idx <- which.max(abs_coefs)
```

```
    best_predictor <- rownames(coefs_no_intercept)[best_idx]
```

```

    best_coef <- coefs_no_intercept[best_idx, 1]
    best_pval <- coefs_no_intercept[best_idx, 4]
  } else {
    best_predictor <- "none"
    best_coef <- NA
    best_pval <- NA
  }
} else {
  best_predictor <- "none"
  best_coef <- NA
  best_pval <- NA
}

# Store results
regression_summary <-<- rbind(regression_summary, data.frame(
  Response_Variable = response,
  Variable_Type = var_type,
  R_squared = round(summary_model$r.squared, 4),
  Adj_R_squared = round(summary_model$adj.r.squared, 4),
  BIC = round(BIC(model), 2),
  AIC = round(AIC(model), 2),
  Num_Predictors = length(coef(model)) - 1,
  Predictors = paste(names(coef(model))[-1], collapse = ", ")

```

```

Best_Predictor = best_predictor,
Best_Predictor_Coef = round(best_coef, 4),
Best_Predictor_Pvalue = round(best_pval, 6),
stringsAsFactors = FALSE
))

cat(" ✓ R-squared:", round(summary_model$r.squared, 4), "\n")
cat(" ✓ BIC:", round(BIC(model), 2), "\n")
cat(" ✓ Predictors:", length(coef(model)) - 1, "\n")

# Save detailed model summary
sink(paste0("2_regression_", response, "_summary.txt"))
cat("Regression Results for:", response, "\n")
cat("=====\n\n")
print(summary_model)
cat("\n\nBIC:", BIC(model), "\n")
cat("AIC:", AIC(model), "\n")
cat("\n\nModel Formula:\n")
print(formula(model))
sink()

} else {
cat(" ✗ ", result$error, "\n")

```

```

    }
  }
}

# Combine all predictors
all_predictors <- unique(c(lipid_vars, perox_vars, antiox_vars, other_vars))

# Analyze each group
if (length(lipid_vars) > 0) {
  cat("\n\n=== ANALYZING LIPID VARIABLES ===\n")
  analyze_response_group(data, lipid_vars, "Lipid", all_predictors)
}

if (length(perox_vars) > 0) {
  cat("\n\n=== ANALYZING LIPID PEROXIDATION VARIABLES ===\n")
  analyze_response_group(data, perox_vars, "Lipid Peroxidation", all_predictors)
}

if (length(antiox_vars) > 0) {
  cat("\n\n=== ANALYZING ANTIOXIDANT VARIABLES ===\n")
  analyze_response_group(data, antiox_vars, "Antioxidant", all_predictors)
}

```

```

# Save regression summary

if (nrow(regression_summary) > 0) {

  regression_summary <- regression_summary[order(regression_summary$BIC), ]

  write.csv(regression_summary,          "2_regression_all_models_summary.csv",
            row.names = FALSE)

  cat("\n\n=====\\n")

  cat("REGRESSION SUMMARY - BEST MODELS BY BIC\\n")

  cat("=====\\n")

  print(head(regression_summary, 10))


# Find best overall model

best_model_idx <- which.min(regression_summary$BIC)

cat("\n★ BEST OVERALL MODEL:\\n")

cat("  Response:", regression_summary$Response_Variable[best_model_idx],
    "\\n")

cat("  R-squared:", regression_summary$R_squared[best_model_idx], "\\n")

cat("  BIC:", regression_summary$BIC[best_model_idx], "\\n")

cat("  Predictors:", regression_summary$Predictors[best_model_idx], "\\n")

}


# =====

# PART 3: LOGISTIC REGRESSION FOR BINARY OUTCOME

```



```
# =====
```

```
cat("\n\n=====\\n")
```

```
cat("PART 3: LOGISTIC REGRESSION FOR OUTCOME\\n")
```

```
cat("=====\\n")
```

```
# Check if outcome column exists
```

```
if (!"outcome" %in% names(data)) {
```

```
  cat("ERROR: No 'outcome' column found in the data!\\n")
```

```
  cat("Available columns:", paste(names(data), collapse=" "), "\\n")
```

```
} else if (length(hormone_vars) == 0) {
```

```
  cat("ERROR: No hormone variables found in the data!\\n")
```

```
} else {
```

```
  # Prepare data for logistic regression
```

```
  logistic_data <- data[complete.cases(data[, c("outcome", hormone_vars)]), ]
```

```
  cat("\nComplete cases for logistic regression:", nrow(logistic_data), "\\n")
```

```
  cat("Outcome distribution:\\n")
```

```
  print(table(logistic_data$outcome))
```

```
  if (nrow(logistic_data) < 10) {
```

```
    cat("ERROR: Insufficient data for logistic regression (need at least 10 complete cases)\\n")
```

```

} else {

# ----- Model 1: Simple logistic regression (each hormone separately) -----
cat("\n-----\n")
cat("Model 1: Simple Logistic Regression\n")
cat("-----\n")

simple_models <- data.frame(
  Hormone = character(),
  Coefficient = numeric(),
  Std_Error = numeric(),
  Odds_Ratio = numeric(),
  CI_Lower = numeric(),
  CI_Upper = numeric(),
  P_value = numeric(),
  AIC = numeric(),
  BIC = numeric(),
  stringsAsFactors = FALSE
)

for (hormone in hormone_vars) {
  model <- glm(outcome ~ get(hormone), data = logistic_data, family = binomial)
  coef_sum <- summary(model)$coefficients

```

```

if (nrow(coef_sum) > 1) {
  coef_val <- coef_sum[2, 1]
  se_val <- coef_sum[2, 2]
  OR <- exp(coef_val)
  CI_lower <- exp(coef_val - 1.96 * se_val)
  CI_upper <- exp(coef_val + 1.96 * se_val)
  p_val <- coef_sum[2, 4]

  simple_models <- rbind(simple_models, data.frame(
    Hormone = hormone,
    Coefficient = round(coef_val, 4),
    Std_Error = round(se_val, 4),
    Odds_Ratio = round(OR, 3),
    CI_Lower = round(CI_lower, 3),
    CI_Upper = round(CI_upper, 3),
    P_value = round(p_val, 4),
    AIC = round(AIC(model), 2),
    BIC = round(BIC(model), 2),
    stringsAsFactors = FALSE
  ))
}
}

```

```

# Sort by BIC
simple_models <- simple_models[order(simple_models$BIC), ]

cat("\nSimple Logistic Regression Results (sorted by BIC):\n")
print(simple_models)

# Highlight significant models
sig_simple <- simple_models[simple_models$P_value < 0.05, ]
if (nrow(sig_simple) > 0) {
  cat("\nSignificant predictors (p < 0.05):\n")
  print(sig_simple[, c("Hormone", "Odds_Ratio", "CI_Lower", "CI_Upper",
"P_value")])
}

# ----- Model 2: Multiple logistic regression (all hormones) -----
cat("\n-----\n")
cat("Model 2: Multiple Logistic Regression (All Hormones)\n")
cat("-----\n")

formula_all <- as.formula(paste("outcome ~", paste(hormone_vars, collapse = "
+ ")))
model_all <- glm(formula_all, data = logistic_data, family = binomial)

```

```

# Calculate odds ratios for all variables

coef_all <- summary(model_all)$coefficients

or_all <- data.frame(
  Variable = rownames(coef_all),
  Coefficient = round(coef_all[, 1], 4),
  Std_Error = round(coef_all[, 2], 4),
  Odds_Ratio = round(exp(coef_all[, 1]), 3),
  CI_Lower = round(exp(coef_all[, 1] - 1.96 * coef_all[, 2]), 3),
  CI_Upper = round(exp(coef_all[, 1] + 1.96 * coef_all[, 2]), 3),
  P_value = round(coef_all[, 4], 4),
  stringsAsFactors = FALSE
)

cat("\nOdds Ratios for All Hormones Model:\n")

print(or_all)

cat("\nModel Fit Statistics:\n")

cat(" AIC:", round(AIC(model_all), 2), "\n")

cat(" BIC:", round(BIC(model_all), 2), "\n")

# ----- Model 3: Stepwise selection using BIC -----

cat("\n-----\n")

```

```
cat("Model 3: Stepwise Logistic Regression (BIC Selection)\n")
```

```
cat("-----\n")
```

```
null_model <- glm(outcome ~ 1, data = logistic_data, family = binomial)
```

```
step_model <- step(null_model,  
  scope = formula_all,  
  direction = "both",  
  trace = 0,  
  k = log(nrow(logistic_data))) # BIC penalty
```

```
selected_vars <- names(coef(step_model))[-1]
```

```
if (length(selected_vars) > 0) {  
  cat("Selected variables:", paste(selected_vars, collapse = ", "), "\n\n")
```

```
# Calculate odds ratios for selected model
```

```
coef_step <- summary(step_model)$coefficients
```

```
or_step <- data.frame(  
  Variable = rownames(coef_step),  
  Coefficient = round(coef_step[, 1], 4),  
  Std_Error = round(coef_step[, 2], 4),  
  Odds_Ratio = round(exp(coef_step[, 1]), 3),
```

```

    CI_Lower = round(exp(coef_step[, 1] - 1.96 * coef_step[, 2]), 3),
    CI_Upper = round(exp(coef_step[, 1] + 1.96 * coef_step[, 2]), 3),
    P_value = round(coef_step[, 4], 4),
    stringsAsFactors = FALSE
  )

  print(or_step)

  cat("\nModel Fit Statistics:\n")
  cat("  AIC:", round(AIC(step_model), 2), "\n")
  cat("  BIC:", round(BIC(step_model), 2), "\n")
} else {
  cat("No variables selected by stepwise procedure\n")
  step_model <- null_model
}

# ----- Model 4: Forward selection using AIC -----
cat("\n-----\n")
cat("Model 4: Forward Selection Logistic Regression (AIC)\n")
cat("-----\n")

forward_model <- step(null_model,
                      scope = formula_all,

```

```

        direction = "forward",
        trace = 0,
        k = 2) # AIC penalty

selected_vars_fwd <- names(coef(forward_model))[-1]

if (length(selected_vars_fwd) > 0) {
  cat("Selected variables:", paste(selected_vars_fwd, collapse = ", "), "\n\n")

  coef_fwd <- summary(forward_model)$coefficients
  or_fwd <- data.frame(
    Variable = rownames(coef_fwd),
    Odds_Ratio = round(exp(coef_fwd[, 1]), 3),
    CI_Lower = round(exp(coef_fwd[, 1] - 1.96 * coef_fwd[, 2]), 3),
    CI_Upper = round(exp(coef_fwd[, 1] + 1.96 * coef_fwd[, 2]), 3),
    P_value = round(coef_fwd[, 4], 4),
    stringsAsFactors = FALSE
  )

  print(or_fwd)

  cat("\nModel AIC:", round(AIC(forward_model), 2), "\n")
  cat("Model BIC:", round(BIC(forward_model), 2), "\n")

```



```

}

# ----- Model Comparison -----

cat("\n=====\\n")

cat("MODEL COMPARISON SUMMARY\\n")

cat("=====\\n")

model_comparison <- data.frame(
  Model = character(),
  AIC = numeric(),
  BIC = numeric(),
  Num_Predictors = integer(),
  stringsAsFactors = FALSE
)

# Add best simple model
if (nrow(simple_models) > 0) {
  best_simple <- simple_models[which.min(simple_models$BIC), ]
  model_comparison <- rbind(model_comparison, data.frame(
    Model = paste("Simple:", best_simple$Hormone),
    AIC = best_simple$AIC,
    BIC = best_simple$BIC,
    Num_Predictors = 1,

```

```

    stringsAsFactors = FALSE
  ))
}

# Add all hormones model
model_comparison <- rbind(model_comparison, data.frame(
  Model = "All Hormones",
  AIC = round(AIC(model_all), 2),
  BIC = round(BIC(model_all), 2),
  Num_Predictors = length(hormone_vars),
  stringsAsFactors = FALSE
))

# Add stepwise BIC model
if (exists("step_model") && length(coef(step_model)) > 1) {
  model_comparison <- rbind(model_comparison, data.frame(
    Model = "Stepwise (BIC)",
    AIC = round(AIC(step_model), 2),
    BIC = round(BIC(step_model), 2),
    Num_Predictors = length(coef(step_model)) - 1,
    stringsAsFactors = FALSE
  ))
}

```

```

# Add forward AIC model

if (exists("forward_model") && length(coef(forward_model)) > 1) {

  model_comparison <- rbind(model_comparison, data.frame(

    Model = "Forward (AIC)",

    AIC = round(AIC(forward_model), 2),

    BIC = round(BIC(forward_model), 2),

    Num_Predictors = length(coef(forward_model)) - 1,

    stringsAsFactors = FALSE

  ))

}


# Add null model

model_comparison <- rbind(model_comparison, data.frame(

  Model = "Null (Intercept only)",

  AIC = round(AIC(null_model), 2),

  BIC = round(BIC(null_model), 2),

  Num_Predictors = 0,

  stringsAsFactors = FALSE

))


# Sort by BIC

model_comparison <- model_comparison[order(model_comparison$BIC), ]

```

```

cat("\nModel Comparison (sorted by BIC - lower is better):\n")
print(model_comparison)

# Best model by BIC
best_idx <- which.min(model_comparison$BIC)

cat("\n★ BEST MODEL by BIC:", model_comparison$Model[best_idx], "\n")
cat(" BIC =", model_comparison$BIC[best_idx], "\n")
cat(" AIC =", model_comparison$AIC[best_idx], "\n")
cat(" Number of predictors =", model_comparison$Num_Predictors[best_idx],
"\n")

# ----- Model Performance Metrics -----

cat("\n===== \n")
cat("MODEL PERFORMANCE METRICS \n")
cat("===== \n")

calculate_performance <- function(model, data, model_name) {
  pred_prob <- predict(model, newdata = data, type = "response")
  pred_class <- ifelse(pred_prob > 0.5, 1, 0)
  actual <- data$outcome

# Confusion matrix

```

```

cm <- table(Predicted = pred_class, Actual = actual)

# Calculate metrics

accuracy <- sum(diag(cm)) / sum(cm)

sensitivity <- NA
specificity <- NA

if (nrow(cm) == 2) {
  sensitivity <- cm[2, 2] / sum(actual == 1)
  specificity <- cm[1, 1] / sum(actual == 0)
}

# Brier score

brier <- mean((pred_prob - actual)^2)

# Pseudo R-squared (McFadden)

null_model <- glm(outcome ~ 1, data = data, family = binomial)
pseudo_r2 <- 1 - (logLik(model) / logLik(null_model))

cat("\n", model_name, ":\n", sep="")
cat(" Accuracy:", round(accuracy, 4), "\n")
cat(" Sensitivity:", round(sensitivity, 4), "\n")
cat(" Specificity:", round(specificity, 4), "\n")

```

```

cat(" Brier Score:", round(brier, 4), "\n")

cat(" Pseudo R-squared:", round(as.numeric(pseudo_r2), 4), "\n")


return(c(Accuracy = accuracy, Sensitivity = sensitivity,
         Specificity = specificity, Brier = brier, Pseudo_R2 =
as.numeric(pseudo_r2)))
}


performance_results <- data.frame(
  Model = character(),
  Accuracy = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric(),
  Brier_Score = numeric(),
  Pseudo_R2 = numeric(),
  stringsAsFactors = FALSE
)


# Evaluate best simple model
if (nrow(simple_models) > 0) {
  best_simple_hormone
simple_models$Hormone[which.min(simple_models$BIC)]
  best_simple_model <- glm(outcome ~ get(best_simple_hormone),

```

```

        data = logistic_data, family = binomial)

perf <- calculate_performance(best_simple_model, logistic_data,
                             paste("Simple:", best_simple_hormone))

performance_results <- rbind(performance_results, data.frame(
  Model = paste("Simple:", best_simple_hormone),
  Accuracy = perf["Accuracy"],
  Sensitivity = perf["Sensitivity"],
  Specificity = perf["Specificity"],
  Brier_Score = perf["Brier"],
  Pseudo_R2 = perf["Pseudo_R2"],
  stringsAsFactors = FALSE
))
}

# Evaluate all hormones model

perf_all <- calculate_performance(model_all, logistic_data, "All Hormones")

performance_results <- rbind(performance_results, data.frame(
  Model = "All Hormones",
  Accuracy = perf_all["Accuracy"],
  Sensitivity = perf_all["Sensitivity"],
  Specificity = perf_all["Specificity"],
  Brier_Score = perf_all["Brier"],
  Pseudo_R2 = perf_all["Pseudo_R2"],

```

```

stringsAsFactors = FALSE
))

# Evaluate stepwise model
if (exists("step_model") && length(coef(step_model)) > 1) {
  perf_step <- calculate_performance(step_model, logistic_data, "Stepwise
(BIC)")

  performance_results <- rbind(performance_results, data.frame(
    Model = "Stepwise (BIC)",
    Accuracy = perf_step["Accuracy"],
    Sensitivity = perf_step["Sensitivity"],
    Specificity = perf_step["Specificity"],
    Brier_Score = perf_step["Brier"],
    Pseudo_R2 = perf_step["Pseudo_R2"],
    stringsAsFactors = FALSE
  ))
}

# Evaluate forward model
if (exists("forward_model") && length(coef(forward_model)) > 1) {
  perf_fwd <- calculate_performance(forward_model, logistic_data, "Forward
(AIC)")

  performance_results <- rbind(performance_results, data.frame(

```



```

Model = "Forward (AIC)",
Accuracy = perf_fwd["Accuracy"],
Sensitivity = perf_fwd["Sensitivity"],
Specificity = perf_fwd["Specificity"],
Brier_Score = perf_fwd["Brier"],
Pseudo_R2 = perf_fwd["Pseudo_R2"],
stringsAsFactors = FALSE
))
}

# Sort by accuracy

performance_results <- performance_results[order(-
performance_results$Accuracy), ]

cat("\n\nPerformance Summary (sorted by Accuracy):\n")
print(performance_results)

# ----- Save all logistic regression results -----

write.csv(simple_models, "3_logistic_simple_models.csv", row.names = FALSE)

write.csv(or_all, "3_logistic_odds_ratios_all_hormones.csv", row.names =
FALSE)

write.csv(model_comparison, "3_logistic_model_comparison.csv", row.names =
FALSE)

```

```
write.csv(performance_results, "3_logistic_model_performance.csv", row.names
= FALSE)
```

```
if (exists("or_step")) {
  write.csv(or_step, "3_logistic_odds_ratios_stepwise.csv", row.names = FALSE)
}
```

```
cat("\n✓ Logistic regression results saved to CSV files\n")
}
}
```

```
# =====
# FINAL SUMMARY REPORT
# =====
```

```
cat("\n\n=====\\n")
cat("FINAL SUMMARY REPORT\\n")
cat("=====\\n")
```

```
# Create comprehensive summary
sink("0_COMPLETE_ANALYSIS_SUMMARY.txt")
```

```
cat("=====\\n")
```

```
cat("COMPLETE ANALYSIS SUMMARY\n")
```

```
cat("=====\n\n")
```

```
cat("DATE:", date(), "\n\n")
```

```
cat("DATA OVERVIEW\n")
```

```
cat("-----\n")
```

```
cat("Total observations:", nrow(data), "\n")
```

```
cat("Total variables:", ncol(data), "\n\n")
```

```
cat("VARIABLES ANALYZED\n")
```

```
cat("-----\n")
```

```
cat("Hormones:", length(hormone_vars), "\n")
```

```
cat("Lipids:", length(lipid_vars), "\n")
```

```
cat("Lipid Peroxidation:", length(perox_vars), "\n")
```

```
cat("Antioxidants:", length(antiox_vars), "\n")
```

```
cat("Other:", length(other_vars), "\n\n")
```

```
# Correlation summary
```

```
cat("=====\n")
```

```
cat("PART 1: CORRELATION ANALYSIS\n")
```

```
cat("=====\n\n")
```

```
cat("Total correlations calculated:", nrow(correlation_results), "\n")
```

```

cat("Significant correlations (p < 0.05):", nrow(sig_cors), "\n")

cat("\nTop 10 strongest correlations:\n")

print(head(correlation_results[, c("Variable1", "Variable2", "Correlation",
"Significance")], 10))

# Regression summary

cat("\n\n===== \n")

cat("PART 2: REGRESSION ANALYSIS\n")

cat("===== \n\n")

if (nrow(regression_summary) > 0) {

  cat("Total regression models fitted:", nrow(regression_summary), "\n")

  cat("\nBest models by variable type:\n")

  for (type in unique(regression_summary$Variable_Type)) {

    type_best <- regression_summary[regression_summary$Variable_Type ==
type, ]

    type_best <- type_best[order(type_best$BIC), ]

    if (nrow(type_best) > 0) {

      cat("\n", type, ":\n", sep="")

      cat(" Best: ", type_best$Response_Variable[1], "\n", sep="")

      cat(" R-squared:", type_best$R_squared[1], "\n")

      cat(" BIC:", type_best$BIC[1], "\n")

```

```
}  
}
```

```
cat("\n BEST OVERALL REGRESSION MODEL:\n")  
best_overall <- regression_summary[which.min(regression_summary$BIC), ]  
cat(" Response:", best_overall$Response_Variable, "\n")  
cat(" R-squared:", best_overall$R_squared, "\n")  
cat(" Adjusted R-squared:", best_overall$Adj_R_squared, "\n")  
cat(" BIC:", best_overall$BIC, "\n")  
cat(" Predictors:", best_overall$Predictors, "\n")  
}
```

```
# Logistic regression summary
```

```
cat("\n\n===== \n")  
cat("PART 3: LOGISTIC REGRESSION\n")  
cat("===== \n\n")
```

```
if (exists("model_comparison") && nrow(model_comparison) > 0) {  
  cat("Total logistic models compared:", nrow(model_com
```